Function Report

Adaptive Traffic Simulator with

Dynamic Rerouting & Signal Control

Every function in the project, what it does, and who calls it.

Project: Adaptive Traffic Simulator

Language: C++ (with Raylib visualization)

Function Report -- Adaptive Traffic Simulator

> *Every function in the project, what it does, and who calls it.*

Table of Contents

1. Call Graph Overview
2. TrafficSignal
3. RoadDetails
4. vehicle\<t\>
5. Graph\<t, size\>
6. Manager\<t, size\>
7. Simulator\<t, size\>
8. Graphics.cpp (main)

Call Graph Overview

> *[Diagram] See the original .md file for the interactive Mermaid call-graph diagram.*

1. TrafficSignal.h

#	Function	What it does	Called by
1	TrafficSignal()	Constructor -- initializes green/red timers to 0, sets minGreenTime=2, queueThreshold=3, maxGreenTime=8, cost weights pa=10, pb=7	RoadDetails constructor (implicitly, as a member)
2	Timer(bool state, float time)	Advances the timer: if green (state=true), increments greentimer and resets redtimer; if red, increments redtimer	Manager::updateSignals() via RoadDetails::light.Timer(), RoadDetails::Time(), RoadDetails::increment()
3	starvationCost() const	Returns redtimer * 3.5f -- a penalty that increases the longer a road has been red, to prevent starvation	Manager::updateSignals() (via light.starvationCost()), TrafficSignal::canSwitch()
4	turnGreen(bool& state)	Sets state = true, resets greentimer = 0	RoadDetails::changeState()
5	turnRed(bool& state)	Sets state = false, resets redtimer = 0	RoadDetails::changeState(), RoadDetails::change_to_red()

#	Function	What it does	Called by
6	canSwitch(int currentQueue, int maxOtherQueue) const	Decides if the currently-green signal can switch away. Returns false if greentimer < minGreenTime. Returns true if mustSwitch(). Prevents switch if the bottleneck ratio > 3. Otherwise switches if current queue < max other queue or starvation > 12	Manager::updateSignals()
7	mustSwitch() const	Returns greentimer >= maxGreenTime -- hard cap forcing a switch	canSwitch()

2. RoadDetails.h

#	Function	What it does	Called by
1	RoadDetails(float ab, float bc, float c, ...)	Constructor -- sets road length, max_speed, capacity, BPR coefficients a (alpha), b (beta), and dischargeCapacity. Initializes vehicles to 0, signal to green	Graph::makeEdge(), Graph::DeleteEdge(), Graph::DeleteVertex()
2	operator<(const RoadDetails& other)	Compares two roads by their calculateWeight() -- used for priority	Implicit in sorted/priority contexts
3	calculateWeight() const	Returns BPR formula travel time: (length/max_speed) (1 + a (vehicles/capacity)^b) -- the core congestion-aware cost	Graph::minimumSpanningtree(), Graph::Shortest_Link_btw_two_vertices(), operator<()
4	DischargeAllowed(int cap, int pop)	Calculates how many vehicles can leave the queue per tick based on available road capacity, queue length, and discharge cap. Returns 0 if signal is red	Manager::entrance(), Manager::entraingfromQueetoEdge(), Manager::arrivalAtIntersection()
5	TimeCal()	Wrapper that calls NonIdealtime() and caches the result in	Not actively called in the current codebase
6	bestTime()	Returns free-flow travel time: length / max_speed	NonIdealtime(), TimeCal(), Manager::recordArrival()
7	NonIdealtime()	Computes congestion-adjusted travel time using BPR formula and stores it in NonIdealWeight	Graph::shortest_Path_btw2_vericex_returing_list(), Graph::shortest_Path_btw2_vericex(), Manager::entrance(), Manager::entraingfromQueetoEdge(), Manager::arrivalAtIntersection(), vehicleEntersRoad(), vehicleExitsRoad(), vehicleJoinsQueue() ... Graphics.cpp
8	vehicleEntersRoad()	Increments currentVehicles (if below capacity), recalculates travel time	Manager::entrance(), Manager::entraingfromQueetoEdge(), Manager::arrivalAtIntersection()
9	vehicleExitsRoad()	Decrements currentVehicles (if > 0), recalculates travel time	Manager::reached(), Manager::arrivalAtIntersection(), Manager::checkBrokenPath()
10	updateVehicles()	Increments queueCount if below capacity -- legacy helper	Not actively called

#	Function	What it does	Called by
11	vehicleJoinsQueue()	Increments queueCount, recalculates travel time	Manager::entrance(), Manager::entraingfromQueetoEdge(), Manager::arrivalAtIntersection()
12	vehicleExitsQueue()	Decrements queueCount (if > 0)	Manager::entrance(), Manager::entraingfromQueetoEdge(), Manager::ShortestPath(), Manager::checkBrokenPath()
13	decVehicles()	Same as vehicleExitsQueue() -- legacy alias	Not actively called
14	Congestion() const	Returns currentVehicles / capacity -- a ratio from 0.0 to 1.0+	Graph::display_edge_of_index(), Graph::AverageRush(), Graph::printRoadSnapshot(), RoadDetails::choosing(), Graphics.cpp heat-map coloring
15	Time(int time)	Forwards to light.Timer()	Not actively called (superseded by direct light.Timer() in updateSignals)
16	changeState()	Toggles signal: calls light.turnRed() if green, light.turnGreen() if red	Manager::updateSignals()
17	change_to_red()	Forces signal to red via	Manager::updateSignals()
18	increment(int time)	Forwards to light.Timer() -- timer helper	Not actively called
19	choosing()	Computes signal-switching cost: pa queueCount + pb Congestion()²	Manager::updateSignals(), Graph::total_System_Cost()

3. vehicle.h

#	Function	What it does	Called by
1	vehicle()	Default constructor -- sets id=0, state to -1 (inactive), all timers to 0	Implicit
2	vehicle(int i, t s, t d)	Parameterized constructor -- sets id, source, destination, current = source, state = 0 (waiting)	Manager::addVehicle()
3	getTakenPath()	Joins selected_path vector into a formatted string like "Karachi -> Sukkur -> Multan"	Manager::reached() (for logging to car_timings.txt)
4	getstart_and_end()	Returns "source->dest" string	Manager::reached() (for logging)

Vehicle State Machine

State	Meaning
-1	Inactive / uninitialized
0	Waiting -- in a queue at an intersection
1	Traveling -- on a road segment, timeRemaining is counting down
2	Arrived -- reached final destination, pending removal

4. Graph.h

#	Function	What it does	Called by
1	Graph()	Default constructor -- undirected graph, Vcount = 0	Not used (directed constructor used instead)
2	Graph(bool x)	Constructor with direction flag. x = true -> directed graph	Simulator::Simulator(), minimumSpanningtree()
3	bfs(int levels[])	Breadth-first search from vertex 0 -- fills levels[] with BFS depth per node, prints visited order	PrintLevels()
4	PrintLevels()	Runs BFS and then prints all vertices grouped by their BFS level	Manager::PrintAllConnetedCityFromAcity()
5	insertVertex(t vertex)	Adds a vertex to the graph array, increments Vcount	Simulator::setupNetwork(), minimumSpanningtree()
6	makeEdge(int a, int b, float len, float speed, float cap)	Creates an edge (road) between vertices a and b with a RoadDetails weight. Adds to both sides if undirected	Simulator::setupNetwork(), minimumSpanningtree()
7	makeEdge(int a, int b, float len, float speed, float cap, float alpha, float	Overloaded version -- also specifies BPR alpha/beta coefficients	Simulator::setupNetwork()
8	getIndex(t label)	Linear search for a vertex label, returns its array index or -1	Used extensively: bfs(), makeEdge() (indirectly), No_of_edges_btw_2_vertices(), DeleteEdge(), DeleteVertex(), dfs(), Shortest_Link_btw_two_vertices(), shortest_Path_btw2_vericex*(), edgeExist(), getDegree(), display_edge_of_index(), Incoming(), getEdgeDetails(), getEdges(), Manager::entrance(), Manager::arrivalAtIntersection(), Manager::entraingfromQueetoEdge()
9	getVertex()	Returns Vcount (number of vertices)	Not actively called
10	isEmpty()	Returns true if Vcount <= 0	Not actively called
11	No_of_edges_btw_2_vertices(t, t)	Counts parallel edges between two specific vertices	Not actively called
12	DeleteEdge(t, t)	Removes an edge between two vertices from the adjacency lists	Not actively called (available for dynamic topology)
13	DeleteVertex(t)	Removes a vertex and all its edges, shifts remaining vertices to fill the gap, updates neighbor indices	Not actively called
14	dfs()	Depth-first search from vertex 0 -- prints traversal order	Not actively called
15	getNodes()	Returns raw pointer to the internal Gnode array	Graphics.cpp -- for drawing edges and city nodes
16	edgeExist(t, t)	Returns true if a directed edge exists from vertex data1 to data2	Not actively called
17	No_Of_Edges()	Counts total edges in the graph (halved for undirected)	Not actively called
18	minimumSpanningtree()	Prim's MST -- builds a new Graph containing only the minimum spanning tree edges	Not actively called

#	Function	What it does	Called by
19	Shortest_Link_btw_two_vertices(t, t)	Finds the minimum-weight direct edge between two vertices (no	Not actively called
20	shortest_Path_btw2_vericex_returning_list(t, t)	* Dijkstra's algorithm (priority-queue optimized) -- returns the shortest path as a list<t> of vertex labels. Uses NonIdealtime() as edge weight (congestion-aware)	Manager::ShortestPath(), Manager::addVehicle()
21	shortest_Path_btw2_vericex(t, t)	Same Dijkstra but prints the path to console instead of returning it	Not actively called
22	searchVertex(t)	Returns true if a vertex with the given label exists	Not actively called
23	getDegree(t)	Returns the degree (number of neighbors) of a vertex	Not actively called
24	Type()	Prints whether the graph is directed or undirected	Not actively called
25	display_edge_of_index(string)	Returns a formatted string with all edges of a vertex including length, vehicles, signal state, congestion	printGrapgh()
26	Incoming(t target)	For directed graphs -- prints all vertices that have an edge pointing	Not actively called
27	getEdgeDetails(t, t)	Returns a reference to the RoadDetails of the edge from vertex data to data1. Throws if not found	Manager::entrance(), Manager::entraingfromQueetoEdge(), Manager::arrivalAtIntersection(), Manager::reached(), Manager::ShortestPath(), Manager::checkBrokenPath(), Manager::recordArrival(),
28	getEdges(t, list<RoadDetails*>)	Returns a list of pointers to all incoming edges to a vertex (edges ending at data)	Manager::updateSignals()
29	AverageRush()	Computes the mean Congestion() across all edges in the graph	Manager::physics(), Simulator::rush(), Graphics.cpp
30	getVertexAt(int i)	Returns the vertex label at index i	Manager::updateSignals(), Graphics.cpp (city positioning + edge
31	printGrapgh()	Returns a full text dump of the graph by calling display_edge_of_index() for every vertex	Manager::printNetwork()
32	total_System_Cost()	Sums choosing() across all edges -- overall network stress metric	Manager::getSystemCost(), Manager::printPerformanceMetrics(), Manager::printCSVRow()
33	printRoadSnapshot()	Returns a formatted string showing only active roads (roads with vehicles or queued vehicles)	Manager::printPerformanceMetrics()

5. Manger.h

#	Function	What it does	Called by

#	Function	What it does	Called by
1	Manager(Graph* m)	Constructor -- stores a pointer to the graph, initializes all counters and	Simulator::Simulator()
2	getVehicleCount()	Returns array_of_vehicles.size()	Not actively called
3	getArrivedCount()	Returns totalArrivedCount	Graphics.cpp (dashboard "COMPLETED" display)
4	getAvgTravelTime()	Returns cumulative average: totalTime / totalArrivedCount	Graphics.cpp (dashboard "AVG" display)
5	getWindowedAvgTravelTime()	Returns the mean of the last windowSize (100) travel times --	Graphics.cpp (dashboard "RECENT" display)
6	getMinTravelTime()	Returns the minimum travel time of any completed vehicle	Graphics.cpp (dashboard "MIN" display)
7	getMaxTravelTime()	Returns the maximum travel time of any completed vehicle	Graphics.cpp (dashboard "MAX" display)
8	getThroughput(float tick)	Returns totalArrivedCount / tick -- vehicles completed per tick	Graphics.cpp (dashboard "THROUGHPUT" display)
9	getTotalDelay()	Returns totalTime - totalFreeFlowTime -- excess time	Graphics.cpp (dashboard "TOTAL DELAY" display)
10	getSystemCost()	Delegates to map->total_System_Cost()	Graphics.cpp (dashboard "SYSTEM COST" display)
11	getRerouteCount()	Returns totalReroutes	Graphics.cpp (dashboard "REROUTES" display)
12	entrance()	Processes vehicles in state 0 (waiting) -- checks signal + discharge capacity. If allowed, moves vehicle onto the road (state -> 1). If blocked, joins the queue	Manager::physics(), Graphics.cpp main loop
13	entraingfromQueetoEdge()	Similar to entrance() -- specifically handles transitioning queued vehicles onto road segments when signals turn green	Manager::physics(), Graphics.cpp main loop
14	getVehicles()	Returns a reference to array_of_vehicles list	Graphics.cpp (for drawing vehicle dots and reading active count)
15	recordArrival(float travelTime, ...)	Records a completed trip -- updates totalArrivedCount, totalTime, min/max travel times, windowed buffer, and accumulates free-flow time for delay calculation	Manager::reached()
16	reached()	Checks each vehicle: if state == 2, logs arrival and erases vehicle. If state == 1 and path has 2 nodes left and timeRemaining <= 0, marks as arrived, writes to car_timings.txt	Manager::physics(), Graphics.cpp main loop
17	ShortestPath()	* Dynamic rerouting -- recalculates Dijkstra for every active vehicle. If a queued vehicle gets a new path, exits the old queue. Counts total reroutes	Manager::arrivalAtIntersection()
18	addVehicle(int id, t src, t dst)	Creates a new vehicle, computes its initial shortest path, adds to array_of_vehicles	Simulator::addMassiveTraffic(), Manager::injectSinusoidalTraffic()

#	Function	What it does	Called by
19	arrivalAtIntersection()	Handles vehicles that finished a road segment (state 1, timeRemaining <= 0, path > 2). Calls ShortestPath() first for rerouting. Exits the current road, enters the next road or queue	Manager::physics(), Graphics.cpp main loop
20	updateSignals()	* Adaptive signal control -- for each intersection, finds all incoming edges, advances timers, computes switching cost (queue cost + starvation). Switches the green signal to the road with the highest cost if conditions allow	Manager::physics(), Graphics.cpp main loop
21	printPerformanceMetrics(float tick, float rush)	Writes a detailed performance report to performance_metrics.txt -- includes arrived count, avg/windowed/min/max/P95 travel times, throughput, delay, system cost, reroutes, active roads	Manager::physics(), Simulator::metric()
22	printCSVHeader()	Writes the CSV column header to metrics.csv	Manager::clearMetricsFile()
23	printCSVRow(float tick, float rush)	Appends one row of metrics data to metrics.csv	Manager::physics(), Simulator::metric()
24	Car_timing_file(float time, int id, ...)	Appends a single vehicle's completion record to car_timings.txt	Manager::reached()
25	clearMetricsFile()	Truncates performance_metrics.txt, car_timings.txt, and map.txt, then writes CSV header	Simulator::clear()
26	printNetwork()	Writes the full graph structure to map.txt	Simulator::addMassiveTraffic()
27	PrintAllConnetedCityFromAcity()	Delegates to Graph::PrintLevels() -- BFS level output	Not actively called
28	checkBrokenPath()	Removes vehicles with empty paths (unreachable destinations). Cleans up road/queue state before erasure	Manager::physics(), Graphics.cpp main loop
29	physics()	* Console-mode simulation loop -- runs the full simulation: injects sinusoidal traffic, advances time, updates signals, moves vehicles, logs metrics every 100 ticks. Runs until all vehicles arrive or 7000-tick timeout	Simulator::run()
30	injectSinusoidalTraffic(int currentTime)	Dynamically adds vehicles using a sine-wave pattern: amplitude (sin(freq t) + 1) + baseline. Randomly picks source from southern cities, destination from northern cities	Manager::physics(), Graphics.cpp main loop

6. Simulator.h

#	Function	What it does	Called by
1	Simulator()	Constructor -- allocates a directed Graph<t,size> and a	Graphics.cpp main()
2	getMap()	Returns pointer to pakistanMap graph	Graphics.cpp (for accessing nodes and edges for drawing)
3	setupNetwork()	* Network topology builder -- inserts 11 Pakistani cities as vertices and creates ~60+ directed road edges with realistic distances, speeds, capacities, and BPR coefficients. Includes main hub (Multan), outer ring, northern cluster, GT road, and long-haul bypasses	Graphics.cpp main()
4	addMassiveTraffic()	Seeds the network with 4,000 randomized vehicles using srand(time(0)), picks random source/destination pairs, then prints the network state	Graphics.cpp main()
5	run()	Starts the console-mode simulation by calling Manager::physics()	Not used in graphics mode (replaced by frame-by-frame stepping in Graphics.cpp)
6	~Simulator()	Destructor -- deallocates cityManager and pakistanMap	Automatic on scope exit
7	metric(float total_time, float x)	Writes both CSV and performance text metrics for a given tick	Graphics.cpp main loop (at tick 1, 50, and every 100 ticks)
8	rush()	Returns map->AverageRush() -- the network-wide average congestion	Graphics.cpp (dashboard rush display, metric logging)
9	clear()	Delegates to Manager::clearMetricsFile() -- resets all output files	Graphics.cpp main() (at startup)

7. Graphics.cpp -- Visual Entry Point

#	Function / Section	What it does	Called by
1	DrawRoadArrowBold(Vector2, Vector2, Color)	Draws a bold directional arrow at the end of a road segment using Raylib triangle primitives	main() -- edge drawing loop
2	GetMetroHeatColor(float rush)	Maps a congestion ratio (0-1) to a color: GREEN -> YELLOW -> ORANGE -> RED	main() -- edge coloring, rush-level bar

#	Function / Section	What it does	Called by
3	main()	* Application entry point -- initializes the simulator, creates the Raylib window in fullscreen, positions cities in a hub-and-spoke layout (Multan at center), then runs a frame-by-frame simulation loop that: (a) advances physics at 0.1 ticks/frame, (b) draws road edges with heat colors, (c) draws vehicle dots interpolated along roads, (d) renders a left-panel dashboard with all KPIs	OS / Runtime

Graphics main() -- Simulation Pipeline per Frame

```
1. Inject sinusoidal traffic (every 50 ticks, up to tick 1500)
2. Update traffic signals (adaptive control)
3. Advance vehicle timers (timeRemaining -= 0.1, timespent += 0.1)
4. Process arrivals (reached)
5. Handle intersection transitions (arrivalAtIntersection -> reroute + move)
6. Move queued vehicles onto roads (entraingfromQueetoEdge)
7. Process initial entrance (entrance)
8. Remove broken paths (checkBrokenPath)
9. Log metrics at tick 1, 50, and every 100 ticks
10. DRAW: edges -> vehicles -> cities -> dashboard panel
```

Summary Statistics

Metric	Count
Total Classes/Structs	6 (TrafficSignal, RoadDetails, vehicle, Graph, Manager, Simulator)
Total Functions	~75
Actively Called Functions	~50
Legacy/Unused Functions	~10 (TimeCal, updateVehicles, decVehicles, Time, increment, dfs, edgeExist, No_Of_Edges, isEmpty, getVertex)
Core Algorithm	Dijkstra with priority queue (shortest_Path_btw2_vericex_returing_list)
Output Files	performance_metrics.txt, metrics.csv, car_timings.txt, map.txt

8. Metrics Deep-Dive -- Functions, Output Files & What Each Metric Means

8.1 Metrics Pipeline Overview

> *[Diagram] See the original .md file for the interactive Mermaid call-graph diagram.*

8.2 Metrics Functions Explained

recordArrival(float travelTime, const vector<t>& selectedPath)

File: Manger.h · Called by: reached()

This is the data collection engine -- called every time a single vehicle completes its trip. It does 5 things:

Step	What it does	Variable updated
1	Increments the completed-vehicle counter	totalArrivedCount++
2	Accumulates the vehicle's total travel time	totalTime += travelTime
3	Updates min/max records	minTravelTime, maxTravelTime
4	Pushes travelTime into a sliding window buffer (last 100 arrivals). If buffer exceeds windowSize, the oldest entry is erased	recentTravelTimes vector
5	Walks the vehicle's actual taken path edge-by-edge, sums up the free-flow time (length / max_speed) for each road segment	totalFreeFlowTime += rd.bestTime()

> *[!] IMPORTANT: Step 5 is critical -- it builds the ideal baseline against which Total Delay is measured. Without it, you can't know how much of a vehicle's travel time was caused by congestion vs. just the physical distance.*

---

printPerformanceMetrics(float tickTime, float averageRush) -- Full Walkthrough

File: Manger.h · Return type: bool
Called by: physics() (every 100 ticks), Simulator::metric() (at tick 1, 50, and every 100 ticks in graphics mode)

Parameters:

tickTime	float	The current simulation tick (e.g. 100, 200, 300...)
averageRush	float	The network-wide average congestion ratio (0.0-1.0+), passed in from Graph::AverageRush()

Step-by-Step Code Breakdown

Step 1 -- Open the file in append mode (Line 411)

```
ofstream outfile("performance_metrics.txt", ios::app);
```

Opens performance_metrics.txt in append mode (ios::app), so each call adds a new report block at the end without erasing previous ones. If the file can't be opened, it prints an error to cerr and returns false.

Step 2 -- Write the report header (Lines 413-415)

```
outfile << "\n--- PERFORMANCE METRICS REPORT ---" << endl;
outfile << "Tick: " << (int)tickTime << endl;
```

Prints a separator and the current tick number. The tick is cast to int to remove the decimal.

Step 3 -- Check if any vehicles have arrived (Line 417)

```
if (this->totalArrivedCount > 0) {
```

All the travel-time metrics only make sense if at least one vehicle has finished its trip. If none have arrived yet, it skips to writing "No vehicles have finished yet." instead.

Step 4 -- Compute Cumulative Average Travel Time (Lines 418-424)

```
float actualAvgTravelTime = (float)this->totalTime / this->totalArrivedCount;
```

- totalTime = sum of timespent for every completed vehicle
- totalArrivedCount = how many vehicles have completed their trip
- Result: the lifetime average travel time across all completed vehicles
- Why it matters: This is the big-picture efficiency metric. It smooths out over time, so spikes in congestion only move it slowly.

Step 5 -- Compute System Throughput (Line 420)

```
float throughput = (float)this->totalArrivedCount / tickTime;
```

- Result: vehicles completed per tick -- a rate metric
- Example: 1234 vehicles arrived by tick 500 -> throughput = 2.47 vehicles/tick
- Why it matters: If throughput drops while active vehicles stays high, the network is gridlocked

---

Step 6 -- Compute Windowed Average Travel Time (Lines 427-435)

```
float windowSum = 0;
for (size_t i = 0; i < recentTravelTimes.size(); i++)
    windowSum += recentTravelTimes[i];
float windowedAvg = windowSum / recentTravelTimes.size();
```

- recentTravelTimes is a sliding window buffer holding the last 100 completed trip times (managed by recordArrival())
- Sums all values in the buffer and divides by its size
- Why it matters: Unlike the cumulative average, this reacts quickly to current conditions. If the network suddenly congests, this metric rises immediately while the cumulative average barely moves. It's like comparing a "last 5 minutes" average vs. a "since the start" average.

Step 7 -- Write Min and Max Travel Times (Lines 438-439)

```
outfile << "Min Travel Time: " << minTravelTime << " ticks" << endl;
outfile << "Max Travel Time: " << maxTravelTime << " ticks" << endl;
```

- minTravelTime -- the fastest trip any vehicle ever completed (updated in recordArrival())
- maxTravelTime -- the slowest trip any vehicle ever completed
- Why it matters: Shows the full range of the network's performance. A huge gap between min and max indicates inconsistent service -- some vehicles zip through while others get stuck.

Step 8 -- Compute P95 (95th Percentile) Travel Time (Lines 440-447)

```
if (recentTravelTimes.size() >= 20) {           // Only if we have enough samples
    vector<float> sorted = recentTravelTimes;     // Copy the buffer
    sort(sorted.begin(), sorted.end());           // Sort ascending
    int p95idx = (int)(sorted.size() * 0.95);     // Find the 95% index
    if (p95idx >= (int)sorted.size())
        p95idx = (int)sorted.size() - 1;          // Clamp to last element
    outfile << "P95 Travel Time: " << sorted[p95idx] << " ticks" << endl;
}
```

- What it does: Copies the recent-times buffer, sorts it, and picks the value at the 95th percentile position
- Example: With 100 recent trips sorted, p95idx = 95, so it picks the 96th-slowest trip
- Why 95th percentile? This is the standard "worst reasonable case" metric used in industry (networking, web performance, transport). It tells you: "95% of recent vehicles arrived faster than this value." It's better than max because max can be a single outlier; P95 shows the typical worst case.
- Guard: Requires at least 20 samples before computing (small samples give meaningless percentiles)

Step 9 -- Compute Total Delay (Lines 452-454)

```
float totalDelay = this->totalTime - this->totalFreeFlowTime;
```

- totalTime = sum of actual travel times for all completed vehicles
- totalFreeFlowTime = sum of bestTime() (= length / max_speed) for every road segment that every completed vehicle actually traveled
- Result: the extra time vehicles spent beyond the ideal. Pure congestion cost.
- Example: If all vehicles together spent 50,000 ticks traveling, but the ideal (empty roads) would have been 30,000 ticks, then Total Delay = 20,000 ticks
- Why it matters: This is the single most important metric for measuring "how much congestion hurt." A delay of 0 = perfect free-flow conditions.

Step 10 -- Write network-level metrics (always, even if no vehicles arrived) (Lines 460-463)

```
outfile << "Total System Cost: " << map->total_System_Cost() << endl;
outfile << "Average Rush Level: " << averageRush << endl;
outfile << "Total Reroutes: " << totalReroutes << endl;
outfile << "Active Vehicles: " << array_of_vehicles.size() << endl;
```

Metric	Source	Meaning
Total System Cost	Graph::total_System_Cost() -> sums choosing() on every edge	choosing() = pa x queueCount + pb x Congestion². This aggregates queue pressure + congestion across the entire network into one number. High = stressed, near 0 = calm
Average Rush Level	Passed in as parameter (from Graph::AverageRush())	Mean of currentVehicles / capacity across all edges. 0.0 = empty, 0.5 = half-loaded, >=1.0 =
Total Reroutes	totalReroutes counter	Incremented in ShortestPath() every time Dijkstra gives a vehicle a different path than it currently has. Shows how actively the system is adapting
Active Vehicles	array_of_vehicles.size()	Vehicles still traveling or waiting. Should trend toward 0 as the simulation progresses

---

Step 11 -- Write the Active Roads Snapshot (Lines 465-469)

```
string roadSnap = map->printRoadSnapshot();
if (!roadSnap.empty()) {
    outfile << "Active Roads:" << endl;
    outfile << roadSnap;
}
```

Calls Graph::printRoadSnapshot() which iterates every edge and only includes roads that have currentVehicles > 0 or queueCount > 0. Each line shows:

```
  Karachi -> Sukkur | Vehicles: 12/80 | Queue: 3 | Congestion: 15% | Signal: GREEN
```

This gives you a snapshot of where traffic actually is at this tick -- which roads are busy, which have queues building up, and which signals are green.

Step 12 -- Return value (Line 472)

```
return (averageRush == 0);
```

Returns true only if the network is completely empty (rush level = 0). This is used by the simulation loop to detect when the network has fully cleared.

Example Output Block

```
--- PERFORMANCE METRICS REPORT ---
Tick: 500
Total Vehicles Arrived: 1234
Cumulative Avg Travel Time: 42.7 ticks
Windowed Avg Travel Time (last 100): 38.2 ticks
Min Travel Time: 3.6 ticks
Max Travel Time: 210.4 ticks
P95 Travel Time: 105.3 ticks
System Throughput: 2.47 vehicles/tick
Total Delay (actual - free_flow): 18420 ticks
Total System Cost: 5830.2
Average Rush Level: 0.32
Total Reroutes: 892
Active Vehicles: 567
Active Roads:
  Karachi -> Sukkur | Vehicles: 12/80 | Queue: 3 | Congestion: 15% | Signal: GREEN
  Multan -> Lahore  | Vehicles: 45/80 | Queue: 12 | Congestion: 56% | Signal: RED
---------------------------------
```

> *[i] NOTE: This function writes to the text file only -- the equivalent CSV data is written by printCSVRow(), which computes all the same metrics but outputs them as a single comma-separated line for easy graphing/analysis.*

---

printCSVHeader()

File: Manger.h · Called by: clearMetricsFile()

Writes the header row to metrics.csv (truncates any existing data):

```
tick,arrived,cumulative_avg,windowed_avg,min_time,max_time,p95_time,throughput,rush_level,system_cost,total_delay,active_veh
```

---

printCSVRow(float tickTime, float averageRush)

File: Manger.h · Called by: physics() (every 100 ticks), Simulator::metric()

Appends one row of comma-separated metrics to metrics.csv. This is the machine-readable companion to printPerformanceMetrics(). Computes all the same values inline.

Car_timing_file(float time, int id, string path, string initial_and_final)

File: Manger.h · Called by: reached()

Appends a single line for one vehicle to car_timings.txt:

```
Vehicle 42      (Karachi->Lahore)       arrived in 67.3 ticks.       Path: Karachi -> Sukkur -> Multan -> Lahore
```

This is called only once per vehicle, at the moment it reaches its destination.

---

clearMetricsFile()

File: Manger.h · Called by: Simulator::clear()

Truncates all three output files and writes a --- NEW SIMULATION SESSION --- header to each, then calls printCSVHeader() to set up the CSV column names.

8.3 Every Metric Explained

Travel Time Metrics

Metric	Formula	Meaning
Cumulative Avg Travel Time	totalTime / totalArrivedCount	The average time (in ticks) it took all vehicles ever completed to travel from source to destination. Includes waiting at queues, signals, and on congested roads. This is a long-term average that smooths out over time
Windowed Avg Travel Time	sum(recentTravelTimes) / recentTravelTimes.size()	The average travel time of only the last 100 completed vehicles. This is a rolling/sliding window metric that reacts faster to current conditions. If the network gets congested, this rises before the cumulative average does
Min Travel Time	Smallest travelTime seen across all completed vehicles	The fastest trip ever recorded. Represents the best-case performance of the network -- typically a short-distance trip during low congestion
Max Travel Time	Largest travelTime seen across all completed vehicles	The worst-case trip -- typically a long-distance vehicle that hit heavy congestion and multiple red
P95 Travel Time	sorted_recent_times[index at 95th percentile]	The travel time at the 95th percentile of recent trips. 95% of recent vehicles arrived faster than this. Only computed when >=20 samples exist. This is the industry-standard "worst reasonable case" metric -- it ignores extreme outliers while still capturing tail latency

Throughput & Delay Metrics

Metric	Formula	Meaning
System Throughput	totalArrivedCount / tickTime	Vehicles completed per tick -- measures how efficiently the network is moving traffic. A higher value = better. If this drops while active_vehicles stays high, the network is gridlocked
Total Delay	totalTime - totalFreeFlowTime	The extra time all vehicles spent beyond what they would have spent if every road was empty (free-flow). totalFreeFlowTime is the sum of length / max_speed for every road segment every completed vehicle actually traveled. A Total Delay of 0 means no congestion was experienced
Total Reroutes	Counter incremented in ShortestPath() whenever a vehicle's recalculated path differs from its current path	How many times a vehicle was dynamically rerouted to avoid congestion. A high number means the adaptive Dijkstra rerouting is active. This metric proves the system doesn't just plan once -- it reacts to real-time conditions

Network Health Metrics

Metric	Formula	Meaning
Average Rush Level	Graph::AverageRush() = mean of Congestion() across all edges	Network-wide average congestion. Each edge's congestion = currentVehicles / capacity. A rush of 0.0 = no vehicles on any road. 0.5 = roads at 50% capacity on average. >=1.0 = roads are at or over capacity
System Cost	Graph::total_System_Cost() = ? choosing() for every edge	Sums the signal-switching cost function pa x queueCount + pb x Congestion² across all edges. This captures both queuing pressure and congestion penalty. A high value = the network is under stress. Drops toward 0 as vehicles clear out
Active Vehicles	array_of_vehicles.size()	How many vehicles are currently in the system (waiting + traveling). Does NOT include vehicles that have already arrived. Starts at 4000 and decreases. If it stops decreasing, some vehicles may be stuck

Per-Vehicle Logging (car_timings.txt)

Field	Meaning
Vehicle ID	The unique integer identifier for the vehicle
Source -> Dest	The original origin and destination cities
Arrived in X ticks	The vehicle's timespent -- total time from spawn to arrival, including all
Path	The selected_path -- the sequence of cities the vehicle actually traveled through (may differ from the originally planned path due to rerouting)

Active Roads Snapshot (in performance_metrics.txt)

Field	Meaning

Field	Meaning
Vehicles: X/Y	currentVehicles / capacity -- how many vehicles are on the road vs. its maximum
Queue: N	queueCount -- vehicles waiting at the intersection to enter this road
Congestion: N%	Congestion() x 100 -- the load ratio as a percentage
Signal: GREEN/RED	signalState -- whether the traffic signal is currently allowing vehicles onto this road

8.4 CSV Column Reference (metrics.csv)

Column	Type	Source	Description
tick	int	Frame counter	Current simulation time step
arrived	int	totalArrivedCount	Cumulative vehicles that have completed their trip
cumulative_avg	float	totalTime / totalArrivedCount	Lifetime average travel time
windowed_avg	float	Rolling buffer mean	Average travel time of last 100 completed vehicles
min_time	float	minTravelTime	Fastest trip ever recorded
max_time	float	maxTravelTime	Slowest trip ever recorded
p95_time	float	95th percentile of rolling buffer	Tail-latency indicator
throughput	float	totalArrivedCount / tick	Completion rate (vehicles/tick)
rush_level	float	Graph::AverageRush()	Network-wide mean congestion ratio
system_cost	float	Graph::total_System_Cost()	Aggregate signal cost function
total_delay	float	totalTime - totalFreeFlowTime	Extra time spent due to congestion
active_vehicles	int	array_of_vehicles.size()	Vehicles still in the system
reroutes	int	totalReroutes	Cumulative path re-calculations

> *[TIP] TIP: You can import metrics.csv directly into Excel or Python (pandas) to graph these metrics over time and visualize how the adaptive signal control and dynamic rerouting improve network performance as the simulation progresses.*